

ALIGNEMENT, SCORE PAR L'ALGORITHME DE NEEDLEMAN-WUNSCH

Mercredi 7 janvier 2026

AKKAYA Tolunay
tolunay.akkaya@etu.univ-orleans.fr
Université d'Orléans

PAUL Nicolas
nicolas.paul1@etu.univ-orleans.fr
Université d'Orléans

Introduction

Ce rapport présente le détail d'un travail réalisé dans le cadre du cours de Programmation parallèle à l'Université d'Orléans. Le projet porte sur l'implantation et l'utilisation de l'algorithme de Needleman-Wunsch (Needleman, 1970) ainsi que sa parallélisation grâce à OpenMP (Dagum, 1998) en langage C++ (ISO/IEC, 2011). Ce document commence par présenter notre travail de programmation et son analyse avant de discuter des résultats obtenus.

Méthodologie

La méthodologie présentée dans cette section décrit notre approche à implanter l'algorithme les questions du sujet. Nous détaillons tout d'abord l'implantation de l'algorithme de Needleman-Wunsch et sa parallélisation. Puis, nous précisons les conditions suivies pour l'analyse de performance.

Pour implanter et paralléliser l'algorithme de Floyd-Warshall, nous avons suivi l'algorithme comme décrit sur le lien indiqué dans le sujet. La parallélisation s'effectue en affectant dynamiquement à un thread disponible une combinaison de deux séquences. Le score obtenu pour cette combinaison vient remplir notre matrice qui sera l'entrée à la suite du programme (Floyd-Warshall, &c.). Le détail complet est disponible dans le fichier source `needleman.cpp` déposé avec ce rapport.

Pour analyser les performances de notre code, nous avons effectué des mesures systématiques en répétant chaque exécution une dizaine de fois, puis calculer la moyenne des temps observés afin de réduire l'impact des fluctuations liées au système. Les mesures temporelles reposent sur la commande UNIX `time`, dont nous avons retenu la valeur de *real time*, correspondant au temps total écoulé durant l'exécution parallèle. L'ensemble des tests a été réalisé sur une machine du laboratoire (salle E11), servant de référence commune : un ordinateur y est équipé d'un processeur Intel Core i7-10700 à

2,90 GHz avec 8 cœurs physiques et 16 threads, et fonctionne sous Linux 5.15.0 avec Ubuntu 22.04.1 LTS. Les codes sources ont été compilés avec le G++ Compiler en version 11.2.0.

Résultats

Dans cette section, nous présentons les performances de notre implantation avec une parallélisation à gros grain et à grain fin sur 2000 séquences ARN. Nos résultats consistent à comprendre l'évolution du temps de calcul du programme selon le nombre de noeuds OpenMPI et de threads OpenMP avec un jeu de données important.

# noeuds	1	2	4	8	16
# threads	16	8	4	2	1
temps (secondes)	78	28	11	19	25

Fig. 1 Évolution du temps d'exécution.

Nous pouvons constater que le temps le plus court est atteint lorsque nous distribuons équitablement les coeurs de la machine. À l'inverse, nous obtenons le temps le plus lent avec un noeud et tous les threads. Cependant, nous gardons un temps « acceptable » si nous diminuons le nombre de thread au profit du nombre de noeuds. Cela est causé par la dépendance du programme sur la communication : seul le séquençage par l'algorithme de Needleman-Wunsch nécessite un parallélisme *fork-join* afin de la matrice des scores (couplage entre les séquences). Quant au reste du programme, le couplage faible entre les distances permet d'exploiter plusieurs noeuds.

En conclusion, nos observations montrent qu'il existe un lien entre le temps d'exécution et la méthode de parallélisation choisie. Ainsi, il est préférable de choisir une architecture à mémoire partagée lorsqu'il existe un fort couplage entre les données à traiter (et donc éviter une duplication inutile et coûteuse) là où une architecture à mémoire distribuée est plus efficace pour des quantités importantes de données faiblement couplées.

Conclusion

Ce projet de programmation parallèle a permis d'explorer les différences entre les deux architectures parallèles (mémoire partagée ou distribuée) et de comprendre leurs avantages et inconvénients selon le couplage des données utilisées, notamment dans le cadre du séquençage et analyse d'ARN, où les deux situations se présentent et nécessitent un procédé capable d'exploiter une mémoire partagée avant de distribuer les données sur différents noeuds. En général, il est important de maîtriser les deux méthodes afin de passer de l'une à l'autre aux besoins de l'application.

Références

Needleman, Saul B. & Wunsch, Christian D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. Dans *Journal of Molecular Biology*. 48 (3), 443–53.

Dagum, L., & Menon, R. (1998). OpenMP: an industry standard API for shared-memory programming. Dans *IEEE computational science and engineering*, 5(1), 46-55.

ISO/IEC. (2011). ISO/IEC 14882:2011 – Information technology – Programming languages – C++. *International Organization for Standardization*.